CUNEF UNIVERSIDAD

Relational databases · Databases practice

Theory: summary

1. **Information systems**: Information system · Components of an information system · The information system pyramid
2. **Files vs databases**: Problems of file-based systems · What a database adds · Schema, sub-schemas and languages (DDL / DML)
3. **The DBMS**: The DBMS and its architecture · Properties of a DBMS · Actors and roles
4. **Transactions (ACID)**: Transactions and atomicity · Consistency · Isolation · Durability
5. **Abstraction levels**: The ANSI/X3/SPARC architecture · Logical and physical independence
6. **Storage and efficiency**: Files, records and fixed-length records · Variable-length records · Storage media and blocks · Response time: Ts = α + βb
7. **File organization and indexes**: File organizations at a glance · Sequential file organization · Heap file organization · Hash file organization · B+ tree file organization · Clustered file organization · ISAM (Indexed Sequential Access Method) · Choosing a file organization
8. **Data lifecycle**: Data models and why we use them · The data modelling lifecycle

Information systems

Information system

The set of elements, ordered and related to one another, that follow rules and give an organization the information it needs to fulfil its purpose.

- **Completeness**: nothing that is needed is missing, neither data nor functional information such as business rules.
- **Reliability** has three facets: **data accuracy** (stored values and calculations are correct), **availability** (the system is up as much of the time as possible, agreed in an SLA, Service Level Agreement) and **consistency** (the same data look the same everywhere and business rules are applied uniformly).

> **Example.** A **99.999%** availability SLA leaves 0.001% of a year for failures: 0.00001 × 525,600 minutes ≈ **5.26 minutes of downtime per year**. A 99.9% SLA would allow about 8.76 hours.

> **Common mistake.** Treating data and information as synonyms. "Balance = 1,250" is a datum; "this customer has had a balance under 2,000 for six months, so offer a credit card" is information.

Components of an information system

An information system is made of contents, physical equipment, logical equipment and people.

- **Contents**: the data and information, internal and/or external to the organization.
- **Physical equipment (hardware)**: servers, networks, storage. Either **on-premise** (you buy the hardware) or **cloud** (pay as you go).
- **Logical equipment (software)**: architectures, cloud computing and specialized software, e.g. a **multi-layer architecture** with independent layers vs **SOA** (Service-Oriented Architecture, microservices).
- **People (human resources)**: system users, DBA (database administrator), data engineer, data analyst, data architect and data scientist.

> **Example.** An online bookshop: **contents** = catalogue, customers, orders and the publishers' price lists; **hardware** = two cloud database servers rented by the hour; **software** = a web shop, a payments microservice and the DBMS; **people** = customers and staff (users), a DBA, a data engineer who loads the publishers' files every night and an analyst who builds the sales dashboard.

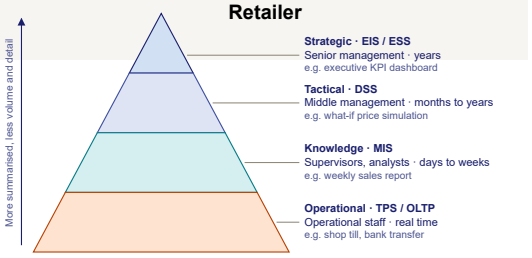
The information system pyramid

The hierarchical levels of information in an organization: the higher the level, the more summarized the information.

Levels of the pyramid, with a bank and a retailer as examples

Level	Volume · detail · use · horizon	Bank	Retailer
Operational (TPS/OLTP)	Very high · maximum · continuous · real time	Records each card payment and ATM withdrawal the instant it happens	Each till sale updates the stock of the store

Level	Volume · detail · use · horizon	Bank	Retailer
Knowledge (MIS)	High · medium · daily/weekly · days/weeks	Daily report of payments and overdrafts per branch	Weekly sales and stock-out report per store
Tactical (DSS)	Medium · low · monthly · months/years	Simulates the effect of raising the mortgage rate 0.5 points	Analyses which products to promote next quarter
Strategic (EIS/ESS)	Low · minimum · quarterly · years	Board dashboard: market share and risk per country	Decides whether to open 20 stores in Portugal



Each level summarizes the one below: fewer, more aggregated data and a longer time horizon as you go up.

> **Example.** One card payment of 42.50 € at a supermarket is a single OLTP row. The MIS adds up the day (1.2 million payments); the DSS compares months; the EIS shows the board one number: card revenue grew 6% this year.

Files vs databases

Problems of file-based systems

Managing an information system with separate files (xlsx, txt, csv...) leads to redundancy, inconsistency, structural dependence, no integrity and limited access.

- **Redundancy**: the same data are repeated in different files.
- **Inconsistency**: one thing is referred to in different ways ("Alberto López" in one file, "A. López" in another), or a copy is updated and the other is not.
- **Isolation of data**: data scattered across files with different formats are hard to combine in one query.
- **Dependence on the file structure**: programs know the exact layout of each file; adding a field means rewriting every program that reads it.
- **No data integrity**: there are no validations, so an age of -20 or a sale of a non-existent product is accepted.
- **Limited access (concurrency)**: only one application or user can modify a file at a time.

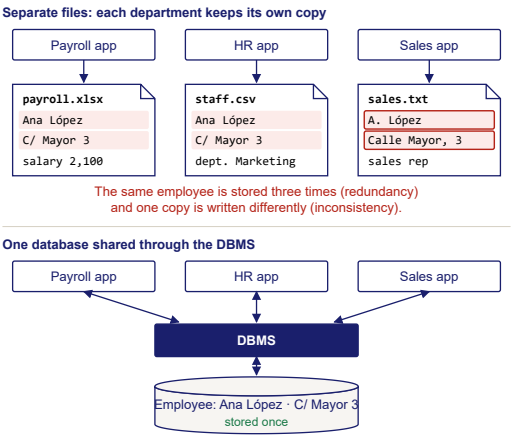
> **Example.** HR keeps `employees_hr.csv` and Payroll keeps `employees_payroll.xlsx`, both with the employee's address. Ana López moves; HR updates its file, Payroll does not. In March her payslip goes to the old address: **redundancy** (two copies) caused an **inconsistency** (two different truths). Joining both files to list "salary per department" needs a hand-written script (**isolation**), and when Payroll adds an IBAN column, the HR report program breaks (**dependence**).

What a database adds

A database is a set of related data that together support the information system of an organization: exhaustive, non-redundant and structured.

- **No redundancy**: each datum is stored once.
- **Consistency**: data are uniform for everybody.
- **Independence**: changes of structure do not affect the programs.
- **Concurrent access**: several users at the same time.
- **Integrity**: automatic validations and relationships between the data.

> **Example.** HR and Payroll now share one table `employees` with one row for Ana López (id 1043). When HR changes her address, Payroll reads the new one in the next query, because there is only one copy. A rule `CHECK (salary > 0)` rejects a negative salary from any application, and both departments can update different employees at the same moment.



Above: each department keeps its own copy of the data. Below: all applications go through one DBMS to a single shared copy.

Schema, sub-schemas and languages (DDL / DML)

The schema is the conceptual organization of the database; sub-schemas are the parts that each user or application sees; DDL defines them and DML works with the data.

DDL commands (structure)

Command	What it does	Syntax
`CREATE`	Creates a table and its columns with their data types	`CREATE TABLE`
`ALTER`	Renames columns, adds or removes a column	`ALTER TABLE`
`RENAME`	Changes the name of the table	`RENAME TABLE`
`COMMENT`	Adds an explanation to the SQL code for other team members	`--` or `/* ... */`
`TRUNCATE`	Deletes the data of a table without deleting the table	`TRUNCATE TABLE`
`DROP`	Deletes the table together with its data	`DROP TABLE`

```
-- DDL: define the structure
CREATE TABLE employees (
  id         INT           PRIMARY KEY,
  name       VARCHAR(50)   NOT NULL,
  department INT,
  salary     DECIMAL(10,2)
);

-- DML: work with the data
INSERT INTO employees (id, name, department, salary)
VALUES (1, 'Ana', 10, 2000.00),
       (2, 'Luis', 10, 2500.00),
       (3, 'Marta', 20, 3000.00);

UPDATE employees SET salary = salary * 1.05 WHERE department = 10;

SELECT name, salary FROM employees WHERE department = 10;
-- Ana 2100.00, Luis 2625.00
```

Example. A sub-schema for the reception app of a gym could be a view with only `name` and `membership_status` of each member: reception never sees bank details. The `UPDATE` above raises only department 10; without `WHERE` it would raise all three employees.

Common mistake. Confusing `TRUNCATE`/`DELETE` with `DROP`. After `TRUNCATE TABLE employees` the empty table is still there; after `DROP TABLE employees` it has to be created again.

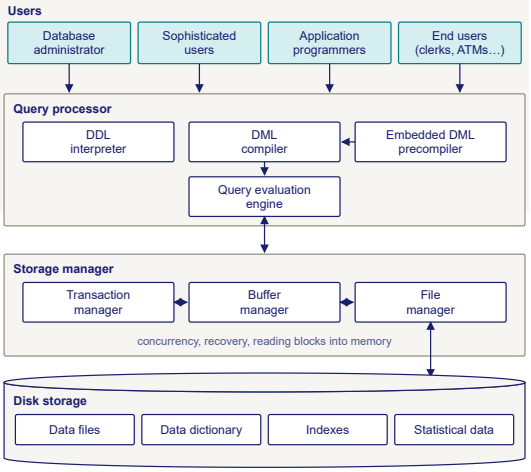
The DBMS

The DBMS and its architecture

A DBMS is a coordinated set of programs, procedures and languages that gives both non-technical users and data specialists the means to describe, retrieve and manipulate data, keeping their integrity, confidentiality and security.

- Security management; installation and configuration of the database; communication management.
- Creation and specification of the data dictionary; creating and administering the physical structure of the database.
- Data manipulation and creation of user applications.
- Export and import of data to and from other systems.
- Creating and restoring backups; recovery in case of disaster.

> **Example.** A cashier's app runs `SELECT balance FROM accounts WHERE account_id = 'A123'`. The DML compiler checks the data dictionary (the table and column exist), uses the statistics to choose the index on `account_id`, the evaluation engine asks the buffer manager for the block, and the file manager reads it from disk only if it is not already in memory.



Users at the top, the DBMS in the middle (query processor and storage manager) and the disk at the bottom (data dictionary, statistics, indexes and data files).

Properties of a DBMS

A DBMS is self-describing, isolates programs from data, offers an abstract view and several views of the data, shares data among users, and provides security and efficiency.

- **Self-describing**: the definition of the database is kept in a special database, the **catalog**, which stores the structure as **metadata**.
- **Program-data isolation**: the structure is separate from the programs, so the database can change without modifying (all) the programs that use it.
- **Abstract view of data**: a conceptual representation that hides storage details and how operations are implemented.
- **Several views of the data**: a subset (a teacher sees only the students enrolled in their subject), a partial view (confidential data hidden by user type) or virtual data derived from others (totals and summaries not stored in the database).
- **Data sharing among many users**: one single store; **transactions** (units of work that group one or more operations and guarantee the ACID properties) control concurrent access.
- **Security** against unauthorized access and system failures.
- **Efficiency**: validations, simultaneous access, etc.

```
-- The catalog is itself queryable (metadata, not user data)
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'employees';

-- A view: a subset plus derived data that is not stored
CREATE VIEW dept_payroll AS
SELECT department, COUNT(*) AS staff, SUM(salary) AS total
FROM employees
GROUP BY department;
```

> **Example.** Querying the catalog for `employees` returns rows such as (`id`, `int`) and (`salary`, `decimal`): data about the data. The view `dept_payroll` returns (10, 2, 4725.00) although no table stores the number 4725.

Actors and roles

Many people work with a database, each through a different door: from end users who only see an app to the DBA who keeps the whole system alive.

Who does what, with a day-to-day example

Role	What they do	A typical day
DBA (database administrator)	Schema changes, physical storage, users and permissions, backups, recovery, tuning	In a hospital, schedules the nightly backup at 02:00, grants the reception app `SELECT` on `Patients` only, and adds an index because the appointments search takes 4 s
Database designer / data architect	Conceptual and logical design; the architect also decides the overall data platform	Draws the ER model of a new loyalty programme: CUSTOMER, CARD, POINTS_MOVEMENT, and decides it goes in the same PostgreSQL cluster as sales
Application developer	Writes programs that embed DML	Codes the "book a class" button of a gym app: `INSERT INTO bookings ...` inside a transaction that also decrements the free places
Data engineer	Builds ETL pipelines that move and transform data between systems	Loads every night the 300,000 card payments of the day from the OLTP database into the data warehouse and fixes the job when a supplier changes its CSV format
Data analyst	Queries and summarizes data for decisions (a sophisticated user)	Writes a `GROUP BY` query showing that sales of umbrellas grew 40% in rainy weeks and puts it in the weekly dashboard
Data scientist	Builds statistical and machine-learning models on the data	Trains a churn model on two years of member visits to predict who will cancel the gym membership
End user	Uses applications with preconfigured queries; never writes SQL	A bank customer checks the balance in the mobile app; a receptionist searches a patient by ID card number

> **Example.** In the hospital above, the receptionist (end user) can read `Patients`, but when she tries to open `Diagnoses` the DBMS refuses, because the DBA granted the reception role only `SELECT` on `Patients`.

> **Common mistake.** Thinking the DBA designs the conceptual model or writes the apps. The DBA owns the schema in production, the physical organization, permissions and maintenance; design and app code belong to other roles.

Transactions (ACID)

Transactions and atomicity

A transaction groups several operations into one unit of work. Atomicity: either all of them are done or none is.

- Step 1: `BEGIN TRANSACTION`. A123 has 5,000 €, B456 has 2,000 €.
- Step 2: the first `UPDATE` debits 1,000 € from A123 (4,000 €, still not committed).
- Step 3: the second `UPDATE` fails: B456 does not exist, the power goes out...
- Step 4: the DBMS rolls back the first `UPDATE`. On restart A123 has 5,000 € again and B456 2,000 €: the money is never debited without being credited.
- If step 3 succeeds, `COMMIT` makes both changes valid together: A123 4,000 €, B456 3,000 €.

```
BEGIN TRANSACTION;
UPDATE accounts SET balance = balance - 1000 WHERE account_id = 'A123';
UPDATE accounts SET balance = balance + 1000 WHERE account_id = 'B456';
COMMIT;

-- Another example: an order, its stock and its invoice
BEGIN TRANSACTION;
INSERT INTO orders (customer_id, total) VALUES (123, 500);
UPDATE inventory SET stock = stock - 5 WHERE product_id = 'P001';
INSERT INTO billing (order_id, amount) VALUES (LAST_INSERT_ID(), 500);
COMMIT;
```

> **Example.** In the order example, if the invoice `INSERT` fails, the order for customer 123 disappears and the 5 units go back to stock: there is never an order without an invoice.

Consistency

The data keep their integrity: a transaction takes the database from one valid state to another, and the DBMS rejects operations that break its rules.

- Rule: `CHECK (balance >= 0)` on `accounts`.
- Step 1: A123 has 600 €. A transfer of 1,000 € to B456 begins.
- Step 2: the debit would leave A123 at -400 €: the DBMS raises an error.
- Step 3: the transaction is rolled back; A123 keeps 600 € and B456 is unchanged. The total money of the bank (600 + 2,000) is the same before and after.

```
-- This transaction FAILS because it breaks the rule "age must be positive"
BEGIN TRANSACTION;
UPDATE students SET age = -20 WHERE student_id = 'A123';
-- ERROR: the database rejects this operation
ROLLBACK;

-- Referential integrity: you cannot delete a customer with active orders
DELETE FROM customers WHERE customer_id = 123;
-- ERROR: violates the foreign key in table orders
```

> **Example.** Customer 123 has two open orders. `DELETE FROM customers WHERE customer_id = 123` is rejected, because otherwise the orders would point to a customer that no longer exists.

Isolation

Transactions do not interfere with one another: a transaction that has not committed is invisible to the rest of the world.

- A123 has 1,000 €. Transfer T1 (300 € to B456) and transfer T2 (500 € to C789) start at the same time.
- Without isolation: T1 reads 1,000, T2 reads 1,000; T1 writes 700, T2 writes 500. Final balance 500 €: T1's debit was **lost** and the bank gave away 300 €.
- With isolation: T2's `UPDATE` on A123 waits (is **blocked**) until T1 commits; then it reads 700 and writes 200. Final balance 200 € = 1,000 - 300 - 500.

```
-- User A
BEGIN TRANSACTION;
SELECT stock FROM products WHERE id = 'P001';   -- sees stock = 1
-- takes a while to decide...
UPDATE products SET stock = 0 WHERE id = 'P001';
INSERT INTO orders (product_id, quantity) VALUES ('P001', 1);
COMMIT;

-- User B, at the same time
BEGIN TRANSACTION;
SELECT stock FROM products WHERE id = 'P001';   -- also sees stock = 1
UPDATE products SET stock = 0 WHERE id = 'P001'; -- BLOCKED until A finishes
-- one of the two transactions will fail for insufficient stock
```

> **Example.** Two customers try to buy the last unit of product P001 at the same instant. Isolation makes the second wait until the first commits, so the last unit is not sold twice (no **overselling**).

Durability

Committed changes are permanent: once a transaction ends with COMMIT, the database cannot lose it.

- Step 1: the transfer of 1,000 € from A123 to B456 executes both `UPDATE`s.
- Step 2: `COMMIT` returns "OK" and the app shows "Transfer completed".
- Step 3: one second later the power goes out; the new balances may not have been written to the data files yet.
- Step 4: on restart the DBMS replays the log: A123 has 4,000 € and B456 3,000 €, exactly what the customer was told.

```
-- Payment confirmation
BEGIN TRANSACTION;
UPDATE orders SET status = 'PAID' WHERE order_id = 12345;
INSERT INTO payments (order_id, amount, date) VALUES (12345, 250.00, NOW());
COMMIT; -- Confirmed!
```

> **Example.** The payment of 250.00 € for order 12345 is still recorded after a power cut that happens right after the `COMMIT`. Before the `COMMIT`, the same power cut would undo it (atomicity).

> **Common mistake.** Thinking that durability means "the data are already in the data files". What it guarantees is that a committed transaction survives any failure; how (log, replicas) is the DBMS's business.

Abstraction levels

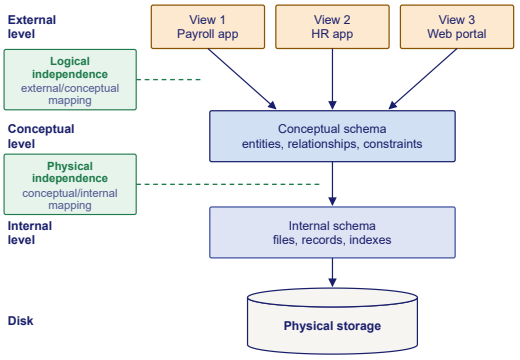
The ANSI/X3/SPARC architecture

The standard, also known as the three-schema architecture, that separates a database into an external, a conceptual and an internal level.

- **External level** (users' view): partial views of the database shown to users and/or applications.
- **Conceptual level** (global logical view): the complete schema; it reflects the structure and relationships of the real-world data to be stored, and isolates the external and internal levels from each other.
- **Internal level** (physical view): specifies what, how and where the data are physically stored on disk.

The three levels for a university database

Level	What it contains
External	Teacher's view: name and mark of the students in her subject. Secretary's view: name, ID card and fees paid, no marks.
Conceptual	`STUDENT(student_id, name, id_card, email)`, `SUBJECT(code, title)`, `ENROLMENT(student_id, code, mark)` with their keys and constraints.
Internal	`ENROLMENT` stored as a B+ tree on `(student_id, code)` in 8 KB blocks, in file `enrol.dat` on an SSD, with an extra index on `code`.



Several external views on top of one conceptual schema, stored by one internal schema. The boundaries between levels give logical and physical independence.

> **Example.** When a teacher opens "My students" she sees 3 columns of 45 rows; she does not know that `ENROLMENT` has 60,000 rows, nor that it lives in a B+ tree on an SSD.

Logical and physical independence

Changing one level does not force changes in the level above: physical independence separates the internal level from the conceptual one; logical independence separates the conceptual level from the external one.

```
-- Physical independence: a new index changes HOW data are stored and found
CREATE INDEX idx_enrol_code ON enrolment (code);
-- The teachers' app still runs exactly the same SELECT, only faster.

-- Logical independence: the conceptual schema grows
ALTER TABLE student ADD phone VARCHAR(20);
-- The view the teachers use does not mention phone, so it keeps working:
CREATE VIEW my_students AS
SELECT s.name, e.mark
FROM student s JOIN enrolment e ON e.student_id = s.student_id
WHERE e.code = 'DB241';
```

> **Example.** Moving `ENROLMENT` from an HDD to an SSD, or adding `idx_enrol_code`, does not change one line of the app (physical). Adding the column `phone` to `student` does not break the view `my_students` (logical).

> **Common mistake.** Swapping the two. Remember: **physical** = internal ↔ conceptual (storage changes); **logical** = conceptual ↔ external (schema changes).

Storage and efficiency

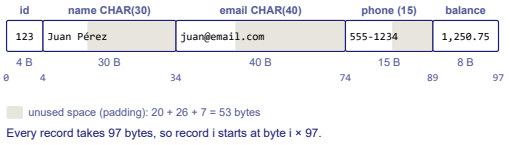
Files, records and fixed-length records

A database is stored as a collection of files; each file is a sequence of records, and each record is made of fields. Records can be of fixed or variable length.

- **Predictable**: all records take the same space.
- **Fast access**: easy to compute the position of any record.
- **Simplicity**: simpler memory management.
- **Waste**: unused space in fields whose content varies.
- **Rigid limits**: a value cannot exceed the maximum size.

Byte layout of a record of `customers_fixed` (97 bytes)

Field	Type	Bytes	Offset
`id`	`INT`	4	0–3
`name`	`CHAR(30)`	30	4–33
`email`	`CHAR(40)`	40	34–73
`phone`	`CHAR(15)`	15	74–88
`balance`	`DECIMAL(10,2)`	8	89–96



Fixed-length records: every field has its maximum width, so every record starts at a multiple of 97 bytes.

```
CREATE TABLE customers_fixed (
  id      INT,            -- 4 bytes
  name    CHAR(30),
  email   CHAR(40),
  phone   CHAR(15),
  balance DECIMAL(10,2)   -- 8 bytes
);
-- Total per record: 97 bytes (always)
```

> **Example.** Record number 1,000 (counting from 0) starts at byte 1,000 × 97 = **97,000**, with no need to read the previous ones. But the name "Ana" uses 3 of its 30 bytes: 27 bytes are padding.

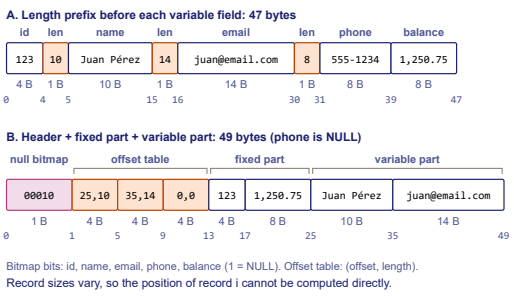
Variable-length records

Fields such as VARCHAR take only the bytes they need plus a length marker, so records have different sizes.

- **Efficiency**: no space is wasted.
- **Flexibility**: fields can grow as needed.
- **Realism**: better for real-world data.
- **Complexity**: harder to manage.
- **Slow access**: the position of a record cannot be computed directly; you must read the lengths (or keep a slot directory).

One record: (7, 'Ana Ruiz', 'ana@cunef.edu', '600123456', 150.00), with a 1-byte length before each VARCHAR

Field	Stored as	Bytes
`id`	7	4
`name`	length 8 + "Ana Ruiz"	1 + 8 = 9
`email`	length 13 + "ana@cunef.edu"	1 + 13 = 14
`phone`	length 9 + "600123456"	1 + 9 = 10
`balance`	150.00	8
Total		**45** (vs 97 in the fixed table)



Variable-length records: each VARCHAR is preceded by its length, so records have different sizes and must be walked to be located.

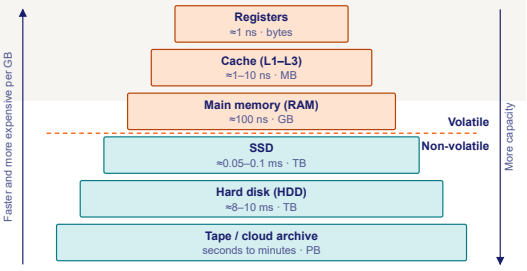
```
CREATE TABLE customers_variable (
  id      INT,            -- 4 bytes
  name    VARCHAR(100),   -- 1-100 bytes + length
  email   VARCHAR(255),   -- 1-255 bytes + length
  phone   VARCHAR(20),    -- 1-20 bytes + length
  balance DECIMAL(10,2)   -- 8 bytes
);
```

> **Example.** The next record, for "Bartolomé Fernández-Villaverde", is longer, so record 2 does not start at a fixed offset. If Ana changes her email to a longer one, her record grows and may have to move.

Storage media and blocks

Databases must live on non-volatile storage; because that storage is slow, data are read and written in large blocks.

> **Example.** With 97-byte records and 4 KB (4,096-byte) blocks, one block holds ⌊4,096 / 97⌋ = **42 records**. Reading 4,200 customers sequentially takes 100 I/Os instead of 4,200.



The closer to the CPU, the faster, smaller, more expensive and volatile the memory; the database lives on the non-volatile levels and is cached in RAM.

Response time: Ts = α + βb

The time to read or write a block is a fixed seek time plus a transfer time proportional to the block size.

Worked example: 10 million transaction records, 100,000 random queries per hour, 2 KB records (b = 0.002 MB)

	HDD (α = 10 ms, 80 MB/s)	SSD (α = 0.05 ms, 300 MB/s)
Time per query	T = 10 ms + $(80\ \text{MB/s})^{-1}$ × 0.002 MB = 10 ms + 0.025 ms ≈ **10 ms**	T = 0.05 ms + $(300\ \text{MB/s})^{-1}$ × 0.002 MB = 0.05 ms + 0.0067 ms ≈ **0.05 ms**
Load per hour	100,000 × 10 ms = 1,000,000 ms = **1,000 s** of disk time per hour	100,000 × 0.05 ms = 5,000 ms = **5 s** per hour
Can it cope?	Viable: uses 1,000 / 3,600 = **27.78%** of the time	Very viable: uses 5 / 3,600 = **0.1389%** of the time

> **Example.** In the worked example the transfer term is tiny: $(80\ \text{MB/s})^{-1}$ × 0.002 MB = 0.000025 s = 0.025 ms against a 10 ms seek. Doubling the record to 4 KB would add only another 0.025 ms on the HDD: that is why reading bigger blocks pays off.

> **Common mistake.** Mixing units. β must be in time per MB (or per KB) and b in the same unit: 4 KB = 0.004 MB, and 1/(100 MB/s) = 0.01 s/MB = 10 ms/MB.

File organization and indexes

File organizations at a glance

The organization of a file is how its information is distributed and located inside it: sequential, heap, hash, B+ tree, clustered or ISAM.

Summary of the six organizations

Organization	Description	Advantages	Drawbacks	Use cases
Sequential	Records stored **one after another** in order; new ones at the end or with full re-sorting	Fast for contiguous data; simple; cheap storage	High read time; deletion fragments; sorting very slow	Backup files, bulk loads, historical data

Organization	Description	Advantages	Drawbacks	Use cases
Heap	**Unordered** records with a unique ID and a bucket address; inserted at the end	Very fast insertion; good for small DBs; no sorting	Slow for large DBs; wastes memory; sequential search	Small DBs, system logs, bulk insertion
Hash	A **hash function** computes the location from the primary key	Very fast direct access; no sorting; constant time	Can delete data by mistake; non-consecutive memory; hash collisions	Medium DBs, access by key, frequent transactions
B+ tree	**Balanced tree**: data in the leaves, internal nodes are navigation pointers	Efficient search; balanced; dynamic; fast traversal	Balancing cost; high complexity; pointer overhead	Large DBs, many updates, sorted queries
Clustered	**Combines several tables** in the same block using a common identifier (index or hash)	Excellent for JOINs; efficient 1:M; fewer disk accesses	Bad for 1:1; not for large DBs; inefficient without JOINs	Frequent 1:M, many JOINs, related tables
ISAM	Separate **index file** that points to the real location of each record	Range searches; search patterns; large DBs	Extra storage for indexes; indexes grow with data; complex maintenance	Very large DBs, complex searches, range queries

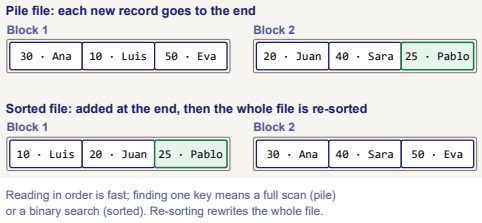
> **Example.** A shop with 2,000 products that are bought by ID: hash. The same shop with 50 million order lines queried by date range: B+ tree. A nightly copy of the whole database that is only ever read from start to end: sequential.

Sequential file organization

Records are stored one after another. In a pile file each new record goes to the end; in a sorted file it is added at the end and then the file is re-sorted.

- **Insert**: pile file, write at the end (cheap); sorted file, append and re-sort the whole file through an auxiliary file (excessively slow).
- **Search**: read records in order until the key appears; on average half the file.
- **Advantages**: fast when accessing contiguous records; simple to implement; data can be kept on cheap storage devices.
- **Drawbacks**: high average read time; deletion can cause internal fragmentation; sorting needs a full pass over all records.
- **Use**: bulk-load files where speed of access and of updates is not critical. Uncommon in DBMSs.

> **Example.** A sorted file of 1,000,000 bank movements ordered by date: printing March is fast (contiguous records), but finding movement 834,201 by its ID means reading about 500,000 records on average, and inserting one movement of 2 January forces rewriting the whole file.



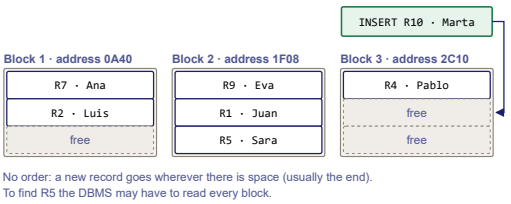
Pile file: each new record goes to the end. Sorted file: it is appended and then the file is re-sorted.

Heap file organization

An unordered file organization: each record has a unique ID linked to a bucket address, and new records are inserted at the end without reordering.

- **Pointers**: each record is linked to its location in memory.
- **No order**: records follow no particular order.
- **Flexibility**: any free block can be used for new records; the DBMS manages allocation, storage and administration automatically.
- **Insert**: append to the end (or any free block), no reorganization: very fast.
- **Search**: scan the file; fast for a small database, slow for a large one.
- **Drawback**: "memory wastage", the gaps left in blocks holding records that do not fill them.
- **Use**: small databases with constant updates or insertions; bulk loads, if there is enough memory.

> **Example.** A web server writes 5,000 log lines per minute into a heap table `access_log`: each line is just appended. Searching "all requests from IP 10.0.0.7" in a table of 200 million lines, though, means reading every block.



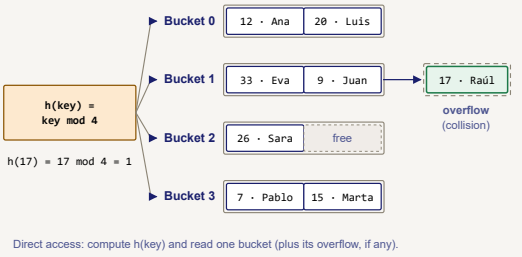
Heap file: records go into any free block in arrival order; each has a bucket address but no order is kept.

Hash file organization

A hash function computes the bucket address of each record from its primary key; the locations it generates are called data buckets or data blocks.

- **Insert**: compute h(key) and write the record in that bucket. The function can be simple or complex; computing it takes constant time, whatever the state of the file.
- **Search by key**: compute h(key) again and go straight to the bucket: no sorting and no scanning.
- **Drawbacks**: accidental deletion if the key is badly chosen (repeated values hash to the same bucket); memory is not used efficiently because records are not consecutive; **collisions**; and no efficient range queries, because neighbouring keys end up in unrelated buckets.
- **Use**: medium databases with constant updates or insertions and access by key.

> **Example.** With 5 buckets and h(id) = id mod 5: employee 1027 goes to bucket 1027 mod 5 = **2**, employee 1030 to bucket **0**. Finding 1027 reads only bucket 2. But "employees with id between 1000 and 1100" must read all 5 buckets. Hashing by `employee_name` would be risky: two employees called "Luis Pérez" fall in the same bucket and a delete by name could remove both; combining the name with the department or the ID card number avoids it.



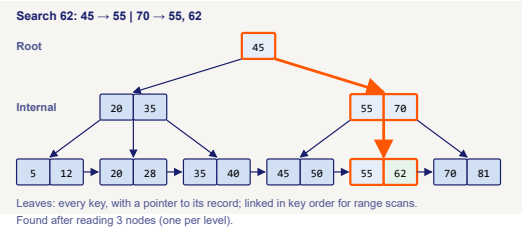
The hash function turns each key into a bucket address; the record is stored and later found in that bucket.

B+ tree file organization

A balanced search tree in which every node can have many children: all records are in the leaf nodes, and internal nodes are pointers that lead to them.

- **Search**: from the root, follow at each level the pointer whose range contains the key, down to a leaf. Cost = height of the tree.
- **Insert/delete**: go down to the right leaf; if it overflows, split it and push a key up. The tree stays **balanced**: all branches have the same depth.
- **Advantages**: efficient search; insertions, deletions and updates do not degrade performance; easy, fast traversal; the size grows or shrinks dynamically.
- **Drawback**: the cost of the balancing operations that keep all branches at the same depth.
- **Use**: large databases with high update rates.

> **Example.** A table of 1,000,000 customers. A full scan compares up to 1,000,000 rows; binary search on a sorted file about $\log_2 1{,}000{,}000 \approx$ **20** steps. A B+ tree with 100 keys per node needs only **3 levels** (100 × 100 × 100 = 1,000,000), that is 3 block reads, and "customers with id between 5,000 and 5,099" continues along the chained leaves.



Internal nodes only guide the search; every record is reached at the same depth, in the linked leaves.

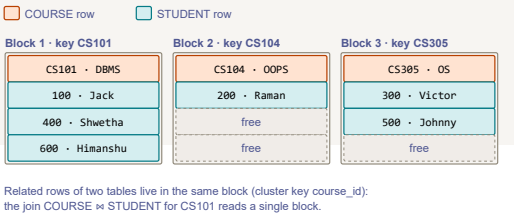
Clustered file organization

Records of two or more tables are combined in the same block according to an identifier/index or the result of a hash function: the cluster ID.

- **Indexed clusters**: the cluster ID is taken directly from the identifier/index value.

- **Hash clusters**: the cluster ID is obtained by applying a hash function to the index.
- **Insert**: the new row goes into the block of its cluster ID, next to its related rows of the other table.
- **Advantages**: good performance for joins between tables; very efficient for 1:M relationships; fewer disk accesses.
- **Drawbacks**: inefficient when joins are not frequent and for 1:1 relationships; not suitable for large databases.
- **Use**: 1:N relationships, e.g. many students can take one course, and frequent joins.

> **Example.** Course DB241 and its 45 students are stored in the same block, keyed by `course_id = DB241`. "List the students of DB241" reads one block instead of one block for the course plus up to 45 scattered blocks for the students.



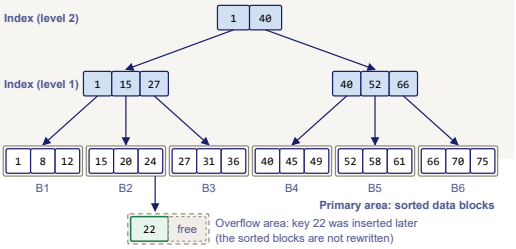
Rows of two tables that share a cluster ID live in the same block, so the join is already done on disk.

ISAM (Indexed Sequential Access Method)

An advanced file organization: from the ID of each record an index entry is created in a separate index file, which points to the bucket address of the block that holds the record.

- **Search**: look up the key (or the start of the range) in the small sorted index file, follow the pointer to the data block, then read on sequentially.
- **Insert**: write the record and add its entry to the index file.
- **Advantages**: makes range searches and search patterns easy.
- **Drawbacks**: extra storage for the index; as records grow, so does the index; more complex maintenance.
- **Use**: large databases, thanks to the efficiency of the indexes for fast access to the data.

> **Example.** "All students whose surname starts with Gar": the index finds the first entry ≥ "Gar" (García, Ana → block 812), and reading on through García, Garrido... stops at the first surname that no longer starts with "Gar". Only a few blocks are read out of thousands.



The index file holds sorted keys with pointers to the data blocks; a range search finds its start in the index and then reads sequentially.

Choosing a file organization

Pick the organization from the size of the database and its main operation.

By main operation (by size: small < 10K records → heap, alternative sequential; medium 10K–1M → hash, alternative B+ tree; large > 1M → B+ tree, alternative ISAM)

Main operation	Best option	Why?
Bulk insertion	Heap file	Does not reorganize, very fast
Search by ID	Hash file	Constant direct access
Sorted searches	B+ tree	Data naturally ordered
Frequent JOINs	Clustered file	Tables physically together
Range searches	ISAM	Optimized indexes
Read only	Sequential file	Simple and efficient

> **Example.** A gym chain: the turnstile log (2 million inserts a day, rarely read) → heap; looking up a member by card number among 300,000 → hash; the 40-million-row table of visits queried "per month" and updated constantly → B+ tree; each class with its bookings, always read together → clustered.

> **Common mistake.** Choosing hash for a table that is mostly queried by ranges ("orders from 1 to 15 May"). Hashing scatters neighbouring keys, so every range query becomes a full scan.

Data lifecycle

Data models and why we use them

A data model is a set of steps, tasks and representation techniques to obtain data structures that solve a problem in a methodical and simple way.

- **Error control**: errors are detected and corrected before implementation.
- **Independence**: data structures are kept separate from the physical and logical environment.
- **Improved maintenance**: the system is easier to manage and update.
- **Better problem understanding**: models make the needs clear.
- **Facilitates communication**: improves collaboration among the members of the development team.

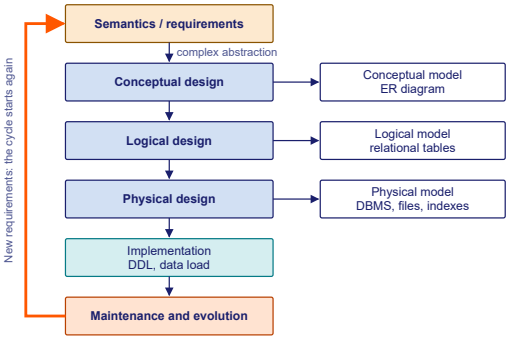
> **Example.** Drawing the model of a gym chain, the team notices that one member can belong to several gyms. Fixing a line in the diagram costs minutes; discovering it after launch, with 300,000 members stored with a single `gym_id`, means migrating data and rewriting queries.

The data modelling lifecycle

From the semantics/requirements, through conceptual, logical and physical design, each producing its model; software engineering techniques loop back from the physical design to the requirements.

Running example: a gym chain, "FitRed"

Stage	What is done	FitRed
Semantics / requirements	Interview users, collect business rules	Members join one or more gyms; gyms offer classes; a member books classes; a class has at most 20 places; staff need visits per month
Conceptual design → conceptual model	E/R diagram, independent of any DBMS	Entities MEMBER, GYM, CLASS; relationship BOOKS (MEMBER M:N CLASS); OFFERS (GYM 1:N CLASS)
Logical design → logical model	Tables, keys, foreign keys	`member(member_id PK, name, email)`, `class(class_id PK, gym_id FK, start, places)`, `booking(member_id FK, class_id FK, PK(member_id, class_id))`
Physical design → physical model	DBMS, types, indexes, file organization	PostgreSQL on SSD; `CHECK (places <= 20)`; B+ tree index on `booking(class_id)`; heap table for turnstile logs
Maintenance (back to requirements)	Monitor, tune, absorb new needs	FitRed adds personal trainers: new entity TRAINER and relationship TRAINS, and the cycle starts again



Requirements → conceptual → logical → physical design, each producing its model; maintenance loops back to the requirements.

> **Example.** In FitRed, the rule "a class has at most 20 places" appears in the requirements, becomes an attribute `places` of CLASS in the conceptual model, a column in the logical model and a `CHECK` constraint plus a transaction in the physical design.